

AquaGuard Hardware Connection Manual

Everything needed to physically wire the AquaGuard ESP32 device: every sensor, both pumps, the servo, power rules, and how it all ties into the code in this folder. Written to be read start to finish before touching a wire, then used as a reference while wiring.

This assumes the ESP32 board itself and the Arduino IDE are already set up (see [hardware/Hardware_Wiring_Guide.pdf](#) if not) — this manual is specifically the *current* wiring + pin map, matching the code in [AquaGuard_v2/](#) as of 2026-08-14 (pH calibration feature, servo step, and Firebase paths all included).

1. Master pin table

Every pin used, in one place. All pins below refer to the ESP32 Dev Module's own silkscreen labels.

Pin	Device	Notes
GPIO 34	pH sensor signal	3.3V only — input-only pin, no over-voltage protection
GPIO 35	TDS sensor signal	3.3V only — same reason as above
GPIO 32	Thermistor	Passive voltage divider, no protection concern
GPIO 5	Ultrasonic TRIG	3.3V logic, no divider needed on this side
GPIO 18	Ultrasonic ECHO	Through a voltage divider — this pin sees 5V from the sensor otherwise
GPIO 25	Pump 1 relay (IN1)	Active LOW, drives an external relay module
GPIO 26	Pump 2 relay (IN2)	Active LOW, drives an external relay module
GPIO 13	Servo signal	Needs external 5V for the servo's own power, not signal

2. Power rail plan — read this before wiring anything

Three separate power concerns, don't mix them up:

- 3.3V-only sensors** (pH, TDS): powered directly from the ESP32's own 3.3V pin. **Never power these from 5V** — GPIO34 and GPIO35 are input-only pins with no over-voltage protection, and a 5V signal on either will risk permanently damaging that pin.
- 5V sensor, protected signal** (ultrasonic): the HC-SR04 itself needs 5V to work reliably, but its ECHO output pin then drives 5V into GPIO18, which can't tolerate that — a resistor voltage divider between ECHO and GPIO18 is mandatory (see §3.3 below), not optional.
- High-current actuators** (both pumps, the servo): powered from their **own external 5V+ supply**, never from the ESP32 board's own 5V/3.3V pins. The ESP32 only shares a common **GND** with these — it never tries to supply their actual operating current. Pumps especially draw far more current than the ESP32's onboard regulator can safely provide; trying to power them from the board risks browning out the whole ESP32 mid-operation (symptom: random resets right when a pump/servo starts moving).

3. Per-device wiring

Wire and test these **one at a time**, in this order, using each device's own standalone sketch in this folder (`01_ph_sensor/` through `06_servo/`) before moving to the next — a wiring mistake shows up immediately in one sketch's Serial output instead of hiding inside the full combined firmware.

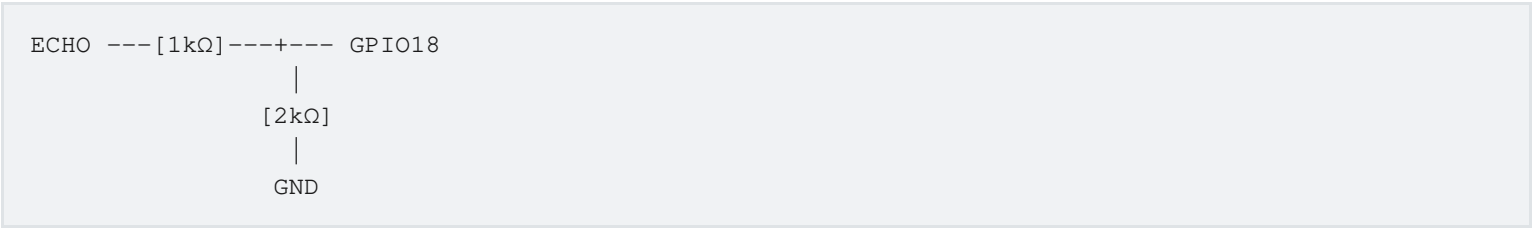
3.1 — pH sensor

pH module pin	Connects to
– (GND)	ESP32 GND
+ (VCC)	ESP32 3.3V
P _o (analog out)	ESP32 GPIO 34
Do, To	Leave unconnected

Test with `01_ph_sensor/01_ph_sensor.ino`. **Calibrate before trusting readings** — run `01_ph_sensor/ph_calibration_tool/ph_calibration_tool.ino` once (or calibrate later through the dashboard's pH calibration page, see `hardware/PH_CALIBRATION_MANUAL.md` — both save to the same place, either works).

3.2 — Ultrasonic (water level)

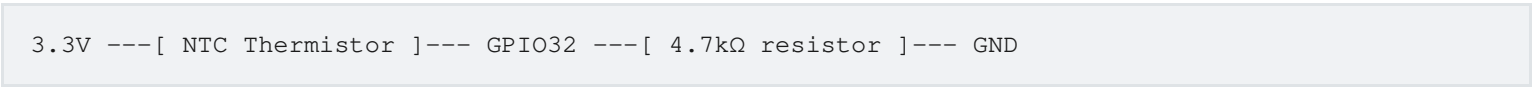
HC-SR04 pin	Connects to
VCC	ESP32 5V
GND	ESP32 GND
TRIG	ESP32 GPIO 5
ECHO	ESP32 GPIO 18 — through the divider below, not directly



Mount pointing straight down at the water surface once installed for real — an angled sensor reflects its echo away from the receiver and reports bad readings.

Test with `02_ultrasonic/02_ultrasonic.ino`.

3.3 — Thermistor (temperature)



- One leg of the thermistor → 3.3V
- Other leg → GPIO32, **and** → one leg of the 4.7kΩ resistor
- Other leg of the resistor → GND

If readings move the *wrong direction* (colder when the room warms up), the thermistor and resistor are swapped — double check which leg goes where.

Test with `03_thermistor/03_thermistor.ino`.

3.4 — TDS sensor (water quality)

TDS module pin	Connects to
VCC	ESP32 3.3V (not 5V — same no-overflow-protection reasoning as the pH sensor)
GND	ESP32 GND
Signal / AOUT	ESP32 GPIO 35

Test with `04_tds/04_tds.ino`.

3.5 — Relay module + both pumps

Relay module pin	Connects to
VCC	External 5V (most relay modules need 5V to switch reliably)
GND	ESP32 GND
IN1	ESP32 GPIO 25 (Pump 1)
IN2	ESP32 GPIO 26 (Pump 2)

Active LOW: GPIO LOW turns the relay ON. If the relay LEDs light when the code thinks it's turning them OFF (or vice versa), your specific module is active-HIGH — swap the logic.

Pumps connect to the relay's OTHER side (COM/NO/NC screw terminals), each from **its own external power supply** — never from the ESP32's own 5V/3.3V. Only a shared GND connects the pump circuit to the ESP32 side.

Check valves: install one on each pump's own outlet/discharge barb, right at the pump — see `hardware/PUMP_CHECK_VALVE_DIAGRAM.pdf` for exactly where and why (stops backflow/siphoning once a pump turns off).

Test with `05_relay_pumps/05_relay_pumps.ino` (relay only, confirm clicking, **before** connecting real pumps to the output side).

3.6 — Servo

Servo wire	Connects to
Signal (orange/yellow)	ESP32 GPIO 13
Power (red)	External 5V — not the ESP32 board
Ground (brown/black)	ESP32 GND (shared ground, power still from the external supply)

If the servo twitches or resets the ESP32 when it starts moving, that's the same current-spike power problem described in §2 — move its power to the same external supply the pumps use.

Test with `06_servo/06_servo.ino`.

4. Software side — what the wiring feeds into

- **Firestore:** the finished device publishes live sensor readings to `/sensor/*`, reads pump commands from `/pumps/*`, and runs the pH calibration protocol over `/phCalibration/*`. Setting up the actual Firestore project (account,

database, security rules) is a one-time step covered in `hardware/FIREBASE_SETUP.md` — do that before or after the physical wiring, doesn't matter which first, but both are needed before `AquaGuard_v2.ino` (the final combined firmware) will fully work.

- **pH calibration:** once wired and the main firmware is flashed, calibrate via the dashboard's `ph-calibration.html` page rather than a separate sketch — see `hardware/PH_CALIBRATION_MANUAL.md`.
- **Pump siphon protection:** a code fix can't solve backflow through an idle pump — that's a physical check-valve installation, §3.5 above and `hardware/PUMP_CHECK_VALVE_DIAGRAM.pdf`.

--	--

5. Bring-up order, start to finish

1. Wire + test §3.1 through §3.6 one at a time, using each step's own sketch in this folder.
2. Install check valves on both pumps' discharge lines (§3.5) — a plumbing step, can happen any time before or during pump testing.
3. Set up the Firebase project (`hardware/FIREBASE_SETUP.md`) — account, database, security rules.
4. Fill in `AquaGuard_v2.ino`'s 2 remaining placeholders (WiFi credentials, Firebase database secret — see this folder's own `README.md`).
5. Upload `AquaGuard_v2.ino` — the real, final, always-running firmware.
6. Calibrate the pH probe through the dashboard (`hardware/PH_CALIBRATION_MANUAL.md`).
7. Confirm live sensor readings and pump control both work end-to-end from the dashboard.

--	--

--	--